Blue-team Analysis of a Cisco Ethical Hacker Final Capstone - Level 1 (Beginner)

1) Project Overview

This project is a beginner-level blue team analysis based on a Cisco Ethical Hacker Final Capstone completed in a controlled lab environment. The capstone was originally designed as an offensive security exercise, however, this report reframes the activity from a defender's perspective.

The purpose of the analysis is to examine how each challenge succeeded, identify the security weakness that enabled it, explain the risks from a blue-team viewpoint, and provide practical remediation recommendations. The report focuses on four challenge areas:

- SQL injection
- Web server misconfiguration
- Exposed Samba shares
- Analysis of a packet capture file

This project is intended to demonstrate beginner-level defensive thinking, structured reporting, and the ability to convert offensive lab work into a professional portfolio artifact.

2) Assessment Scope and Lab Context

- ***Environment***: Controlled educational lab
- ***Analyst system***: Preconfigured Kali Linux virtual machine
- ***Virtualization platform***: Oracle VirtualBox
- ***Networks in scope***: 10.6.6.0/24 and 172.17.0.0/24
- ***Activity type***: Authorized capstone simulation
- ***Goal***: Identify vulnerabilities, exploit them to complete challenge objectives, then analyze the weaknesses from a blue-team perspective.

No real-world systems were targeted. All activity was performed inside the authorized lab environment for training and portfolio development purposes.

3) Capstone Objectives

The goal of this report is to:

- Identify the main weakness involved in each challenge
- Explain how the attacker was able to succeed
- Interpret each action from a defender's point of view
- Summarize the security risks created by these weaknesses
- Recommend practical remediation actions appropriate for a beginner blue-team analysis

4) High-Level Attack Flow

The capstone followed four main challenge paths:

1. SQL injection was used to expose account information, including a username and password hash, which then enabled access to a target system.
2. Web server misconfigurations exposed indexed directories and accessible files through the browser.
3. Open Samba shares allowed anonymous access to files and enabled retrieval of a challenge flag.
4. Packet capture analysis revealed a relevant host and web-accessible data path, showing how network traffic can disclose sensitive information

Overall, the capstone demonstrated that relatively basic weaknesses can provide an attacker with useful intelligence, unauthorized access, and direct exposure to sensitive files.

5) Finding 1 - SQL injection Enabled Credential Exposure

What happened

- A vulnerable input field in the web application allowed SQL injection. By abusing that input, account-related data was returned by the application, including Gordon Brown related information, the username '*gordonb*', and an associated password hash. That hash was then cracked, and the recovered credentials were successfully used to access the target system at 172.17.0.2, where the Challenge 1 flag file was retrieved.

Why it worked

- This attack succeeded because the application did not securely handle user input. Instead of treating input as harmless data, the application allowed it to influence a backend database query. As a result, an attacker could retrieve unauthorized information from the database. The exposed password hash also appears to have been weak enough to be cracked quickly, which enabled follow-on access.

Defender view

- From a blue-team perspective, this finding shows two linked problems:
    - Insecure input handling in the web application
    - Weak credential protection after database disclosure

  A defender should understand that SQL is not only a database issue, it can directly lead to account compromise, lateral movement, and access to additional systems if the exposed data is unable elsewhere.

Risk

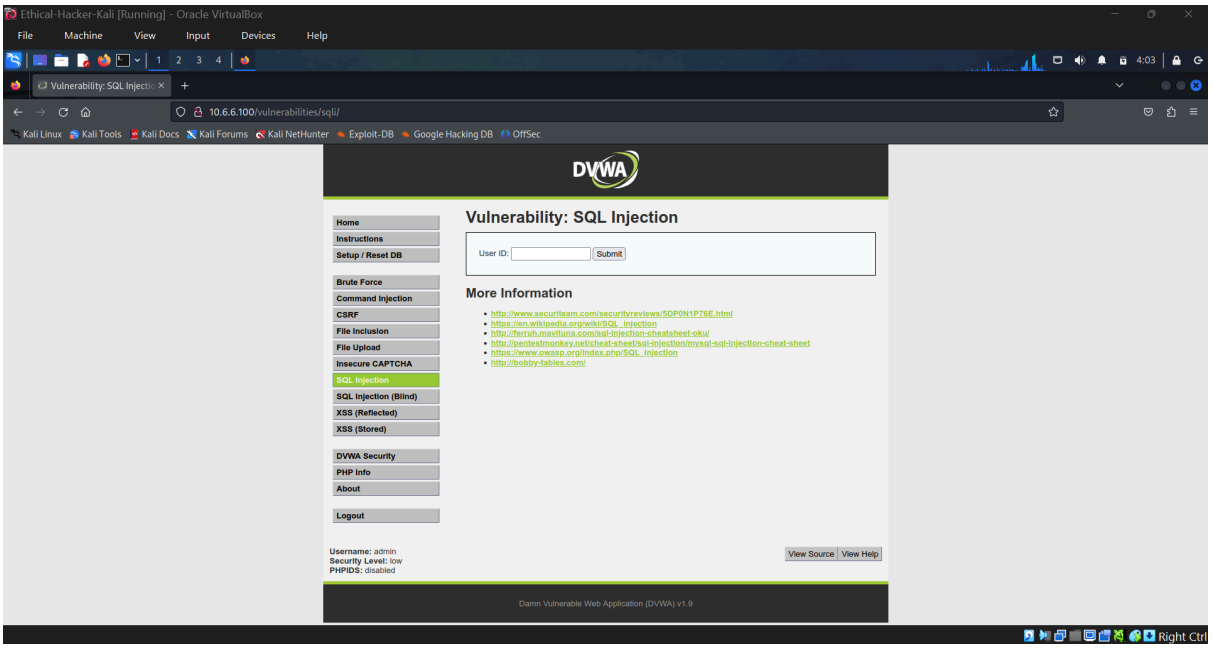
- The main risk is unauthorised disclosure of sensitive database content, especially usernames and password hashes. If those hashes are weak or reused, attackers

may gain valid credentials and pivot into other systems. In a real environment, this could lead to account compromise, unauthorised access, and broader system exposure.

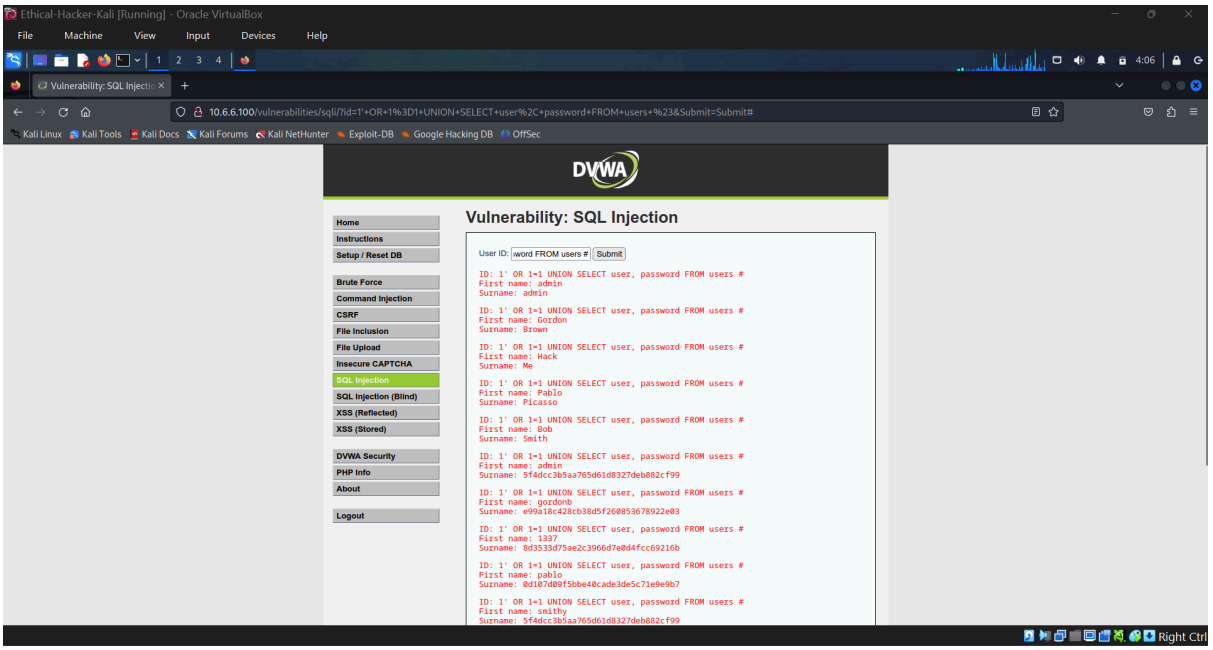
Recommendation

- Use parameterized queries or prepared statements for all database access.
- Validate and sanitize user input appropriately.
- Avoid exposing sensitive database content through application responses.
- Enforce strong password policies.
- Store passwords securely using strong modern hashing methods.
- Monitor for unusual input patterns and repeated attacks against web forms.

Evidence



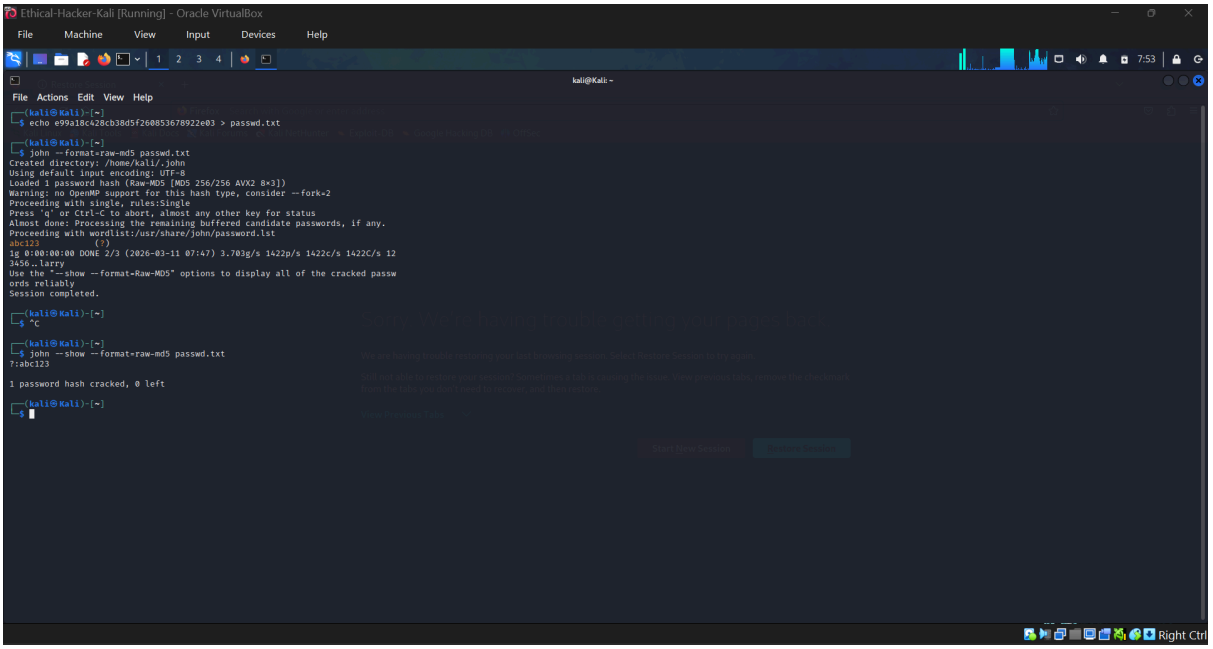
(vulnerable SQL input field identified)



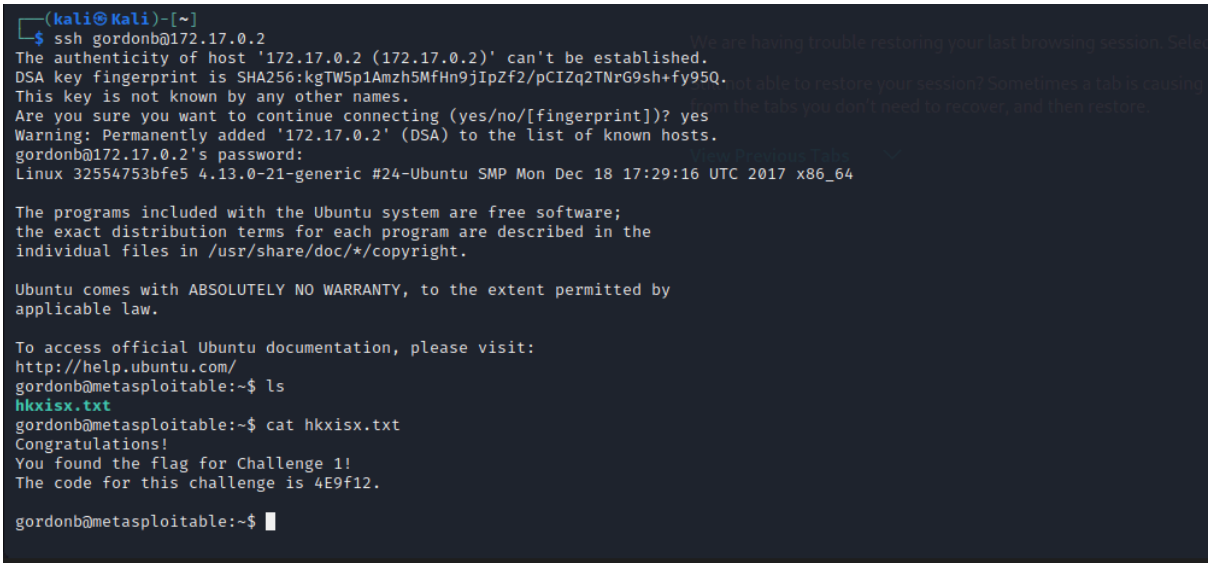
(user data exposed using SQL injection)

```
ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Gordon
Surname: Brown

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Hack
Surname: Me

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Pablo
Surname: Picasso

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: Bob
Surname: Smith

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' OR 1=1 UNION SELECT user, password FROM users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03
```

(Gordon Brown and password hash exposed)



(Gordon Brown password cracked)



(challenge 1 ssh access and flag identified)

6) Finding 2 - Web Server Directory Listing Exposed Sensitive Files

What happened

- Web reconnaissance identified exposed directories on the target web server. Browser-based investigation confirmed that directory indexing was enabled, allowing the contents of '/docs/' to be listed and browsed directly. Continued navigation through the exposed content led to the discovery of the Challenge 2 file and flag.

Why it worked

- This challenge succeeded because the web server was configured in a way that allowed directory listing. Instead of restricting access or disabling indexing, the server exposed file and directory contents to anyone who could reach the path. This made it possible to browse available resources without authentication or deeper exploitation.

Defender View

- From a blue-team perspective, this is a web hardening failure. Even without a complex exploit, an attacker can learn a great deal from exposed directories, including file names, application structure, documentation, and sensitive resources. This kind of exposure helps attackers map the application and identify valuable files quickly.

Risk

- The risk is unauthorised access to files and unnecessary disclosure of internal application structure. Exposed directories can reveal documentation, scripts, configuration clues, and sensitive files that support future compromise. In a real environment, this could lead to information leakage, unauthorized data access, and easier follow-on attacks.

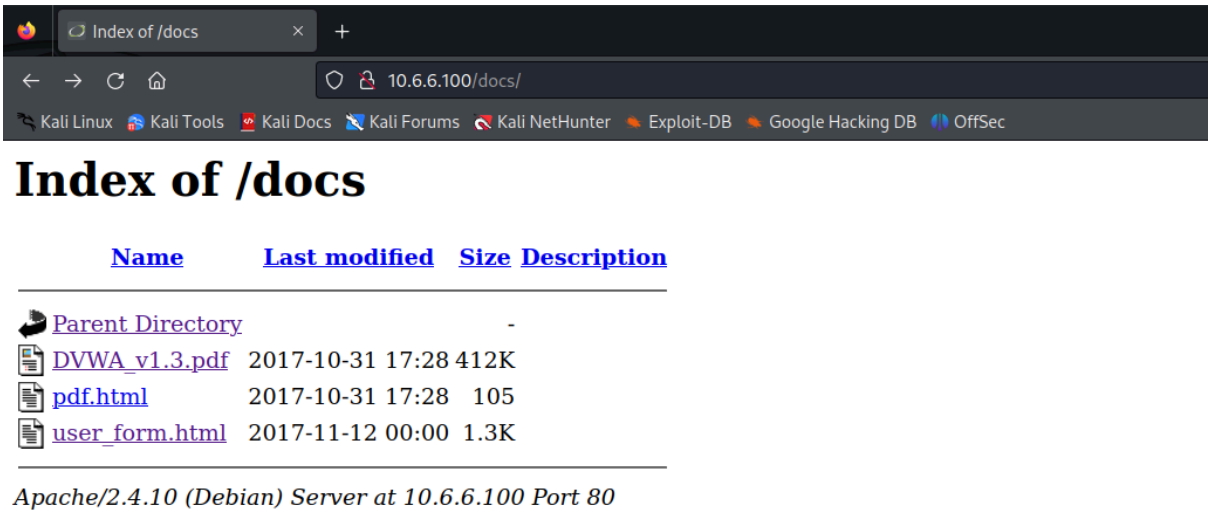
Recommendation

- Disable directory indexing on the web server.
- Restrict direct access to sensitive directories and files.
- Review web server configuration for unnecessary exposure.
- Remove or protect files that do not need to be publicly accessible.
- Perform regular web-content reviews to identify exposed resources.
- Monitor for suspicious directory browsing and repeated enumeration activity.

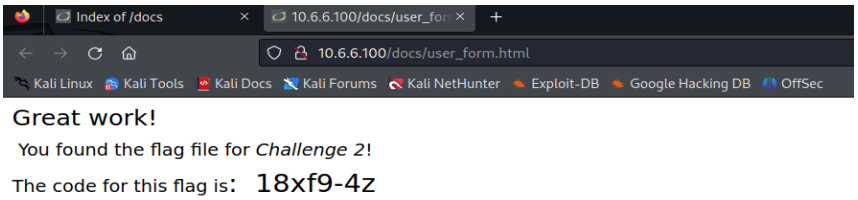
Evidence

```
┌──(kali㉿Kali)-[~]
└─$ nikto -h 10.6.6.100
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          10.6.6.100
+ Target Hostname:    10.6.6.100
+ Target Port:        80
+ Start Time:         2026-03-11 08:04:25 (GMT0)
---------------------------------------------------------------------------
+ Server: Apache/2.4.10 (Debian)
+ /: Cookie PHPSESSID created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/miss
ing-content-type-header/
+ Root page / redirects to: login.php
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Apache/2.4.10 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /config/: Directory indexing found.
+ /config/: Configuration information may be available remotely.
+ /docs/: Directory indexing found.
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ /login.php: Admin login page/section found.
+ 8074 requests: 0 error(s) and 9 item(s) reported on remote host
+ End Time:           2026-03-11 08:04:40 (GMT0) (15 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested

┌──(kali㉿Kali)-[~]
└─$
```

(nikto directory indexing discovered)



(directory listing visible browser)



(Challenge 2 flag visible)

7) Finding 3 - Open Samba Share Allowed Unauthorized File Access

What happened

- Network reconnaissance identified a host with SMB-related services exposed on ports 139 and 445. Further enumeration showed that multiple Samba shares were available and allowed anonymous READ/WRITE access. The accessible share contents were browsed, a file was retrieved, and the Challenge 3 flag was successfully obtained.

Why it worked

- This challenge succeeded because the file-sharing service was configured with overly permissive access controls. Anonymous access to network shares meant an unauthenticated user could list content, move through directories, and retrieve files without proper authorization.

Defender view

- From a blue-team perspective, this is a classic access control failure. File-sharing services are high-value targets because they often contain internal documents, operational data, or user files. If shares are accessible anonymously or with overly broad permission, attackers can retrieve sensitive content with minimal effort.

Risk

- The risk is unauthorized access to internal file shares and exposure of sensitive data. In a real environment, weakly protected SMB shares can lead to document theft, credential harvesting, malware staging, and further internal compromise.

Recommendation

- Disable anonymous access to Samba shares.
- Apply least-privilege permissions to all file shares.
- Review share configurations regularly.
- Remove unnecessary shares and restrict visibility of internal resources.
- Log and monitor access to file shares.
- Segment sensitive internal services from general user access where possible.

Evidence

```
┌──(kali㉿Kali)-[~]
└─$ nmap 10.6.6.0/24
Starting Nmap 7.94 ( https://nmap.org ) at 2026-03-11 08:16 UTC
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 10.6.6.1
Host is up (0.00091s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT   STATE SERVICE
22/tcp open  ssh

Nmap scan report for webgoat.vm (10.6.6.11)
Host is up (0.00095s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT     STATE SERVICE
8080/tcp open  http-proxy
8888/tcp open  sun-answerbook
9001/tcp open  tor-orport

Nmap scan report for juice-shop.vm (10.6.6.12)
Host is up (0.00098s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT     STATE SERVICE
3000/tcp open  ppp

Nmap scan report for dvwa.vm (10.6.6.13)
Host is up (0.00097s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT   STATE SERVICE
80/tcp open  http

Nmap scan report for mutillidae.vm (10.6.6.14)
Host is up (0.00081s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT     STATE SERVICE
3306/tcp open  mysql

Nmap scan report for gravemind.vm (10.6.6.23)
Host is up (0.00070s latency).
Not shown: 994 closed tcp ports (conn-refused)
PORT    STATE SERVICE
21/tcp  open  ftp
22/tcp  open  ssh
53/tcp  open  domain
80/tcp  open  http
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds

Nmap scan report for 10.6.6.100
Host is up (0.00083s latency).
```

(nmap SMB services identified - 1)

```
Nmap scan report for 10.6.6.100
Host is up (0.00083s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT   STATE SERVICE
80/tcp open  http

Nmap done: 256 IP addresses (7 hosts up) scanned in 3.09 seconds

┌──(kali㉿Kali)-[~]
└─$
```

(nmap SMB services identified - 2)

```
┌──(kali㉿Kali)-[~]
└─$ nmap -sV  --script=smb-enum-shares 10.6.6.23
Starting Nmap 7.94 ( https://nmap.org ) at 2026-03-11 08:22 UTC
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for gravemind.vm (10.6.6.23)
Host is up (0.00019s latency).
Not shown: 994 closed tcp ports (conn-refused)
PORT    STATE SERVICE     VERSION
21/tcp  open  ftp         vsftpd 3.0.3
22/tcp  open  ssh         OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
53/tcp  open  domain      ISC BIND 9.11.5-P4-5.1+deb10u5 (Debian Linux)
80/tcp  open  http        nginx 1.14.2
|_http-server-header: nginx/1.14.2
139/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
Service Info: Host: GRAVEMIND; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
| smb-enum-shares:
|   account_used: <blank>
|   \\10.6.6.23\IPC$:
|     Type: STYPE_IPC_HIDDEN
|     Comment: IPC Service (Samba 4.9.5-Debian)
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: READ/WRITE
|   \\10.6.6.23\print$:
|     Type: STYPE_DISKTREE
|     Comment: Printer Drivers
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\var\lib\samba\printers
|     Anonymous access: READ/WRITE
|   \\10.6.6.23\workfiles:
|     Type: STYPE_DISKTREE
|     Comment: Confidential Workfiles
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\var\spool\samba
|_    Anonymous access: READ/WRITE

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.92 seconds

┌──(kali㉿Kali)-[~]
└─$
```

(Samba share enumeration)

```
┌──(kali㉿Kali)-[~]
└─$ smbclient //10.6.6.23/print$
Password for [WORKGROUP\kali]:
Anonymous login successful
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Mon Aug 14 09:40:01 2023
  ..                                  D        0  Mon Aug 30 05:00:05 2021
  IA64                                D        0  Mon Sep  2 13:39:42 2019
  x64                                 D        0  Mon Aug 30 05:00:05 2021
  W32X86                              D        0  Mon Aug 30 05:00:05 2021
  W32MIPS                             D        0  Mon Sep  2 13:39:42 2019
  W32ALPHA                            D        0  Mon Sep  2 13:39:42 2019
  COLOR                               D        0  Mon Sep  2 13:39:42 2019
  W32PPC                              D        0  Mon Sep  2 13:39:42 2019
  WIN40                               D        0  Mon Sep  2 13:39:42 2019
  OTHER                               D        0  Tue Aug 10 00:00:00 2021
  color                               D        0  Mon Aug 30 05:00:05 2021

                38497656 blocks of size 1024. 9078692 blocks available
smb: \> cd OTHER
smb: \OTHER\> get taxes.txt
getting file \OTHER\taxes.txt of size 103 as taxes.txt (1.7 KiloBytes/sec) (average 1.7 KiloBytes/sec)
smb: \OTHER\> exit

┌──(kali㉿Kali)-[~]
└─$ cat taxes.txt
Congratulations!
You found the flag for Challenge 3!
The code for this challenge is A9!15wa2.


┌──(kali㉿Kali)-[~]
└─$
```

(challenge 3 Samba access and flag)

8) Finding 4 - PCAP Analysis Revealed Sensitive Information

What happened

- A packet capture file, '*SA.pcap*', was opened and analysed in Wireshark. HTTP traffic within the capture revealed communication involving host 10.6.6.14 and access to the '/data' path. This indicated a relevant target host and a potentially sensitive web-accessible location. The exact file location associated with the challenge was later confirmed as '*http://10.6.6.14/data/accounts.xml*' when the live resource was unavailable during final documentation.

Why it worked

- This challenge succeeded because network traffic exposed useful information in cleartext. Even without immediately loading the final resource in a browser, the packet capture revealed enough detail to identify a meaningful host and path. This demonstrates how passive traffic analysis can disclose sensitive operational information.

Defender view

- From a blue-team perspective, this shows that traffic itself can be an information source for attackers. Cleartest HTTP requests, visible resources paths, and identifiable internal destinations can reveal system structure and sensitive locations. Attackers do not always need direct exploitation first. Sometimes observation of traffic is enough to guide the next step.

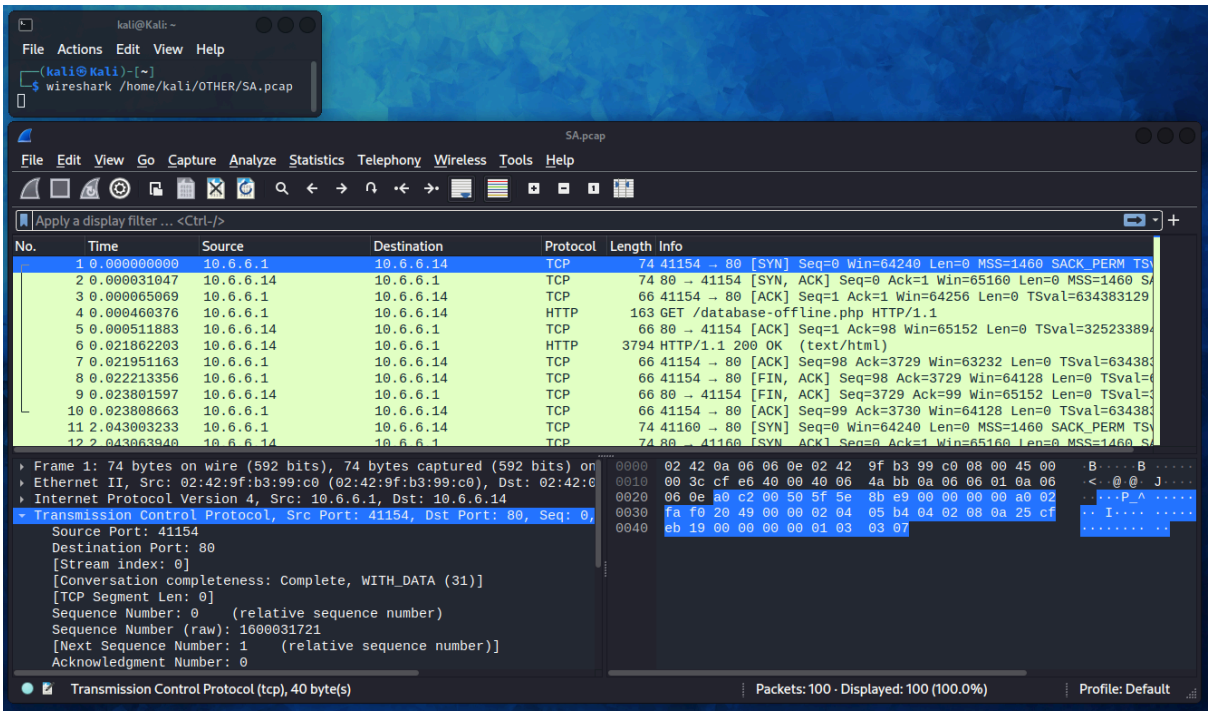
Risk

- The risk is information leakage through network communications. If sensitive paths, files, or internal web resources are visible in traffic, attackers can use that intelligence to target systems more efficiently. In a real environment, this could support reconnaissance, data discovery, and further compromise.

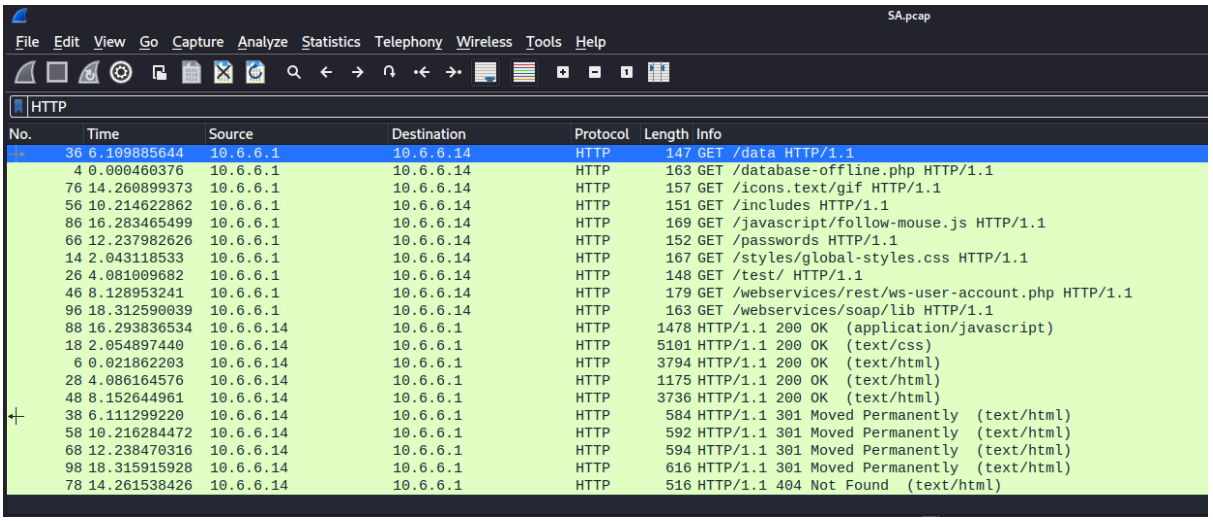
Recommendation

- Reduce unnecessary exposure of sensitive resource paths.
- Protect sensitive communications appropriately.
- Review internal application design to avoid revealing valuable file locations through traffic.
- Restrict access to sensitive data resources.
- Monitor unusual requests to internal data locations.
- Review packet captures and logs during investigations for evidence of resource discovery.

Evidence



(SA.pcap opened in wireshark)



(pcap url and target identified)

The code for Challenge 4 at http://10.6.6.14/data/accounts.xml. The flag value is in the Signature field for the employee ID="0" record.

(challenge 4 answer key confirmation)

Note: The exact Challenge 4 files location was confirmed from the lab guidance because the live web resource was unavailable during final documentation.

9) Key Defensive Weaknesses Observed

The capstone highlighted several broader defensive weaknesses:

- Insecure input handling in web applications
- Weak protection of credential-related data
- Web server misconfiguration allowing directory indexing
- Excessive exposure of file-sharing services
- Anonymous access to internal shares
- Unnecessary disclosure of sensitive paths through network traffic
- Insufficient service hardening and access restriction

These weaknesses are basic, but they are highly important because they enable attackers to progress quickly with relatively little effort.

10) Overall Risk Summary

Overall, the capstone demonstrated that multiple straightforward weaknesses can create meaningful attack paths. The most important pattern across all four challenges was that poor security configuration and weak access control allowed simple attacker actions to produce useful results.

The SQL injection challenge showed how insecure input handling can expose credentials and lead to account compromise. The web-directory challenge showed how misconfiguration alone can expose sensitive files. The Samba challenge showed how anonymous file-share access creates direct data exposure. The PCAP challenge showed that even passive observation of traffic can reveal valuable targets and sensitive paths.

In a real environment, weaknesses like these could contribute to unauthorized access, information leakage, compromised accounts, and broader internal exposure.

11) Recommendations

Based on the findings in this report, the following actions are recommended:

1.) **Secure database-driven web applications**
Use parameterized queries, validate input properly, and avoid exposing backend data through application responses.

2.) **Improve credential security**
Enforce stronger passwords, prevent password reuse, and store passwords using strong, modern hashing methods.

3.) **Harden web server configuration**
Disable anonymous access to Samba shares and apply least-privilege permissions.

4.) **Restrict file-share access**
Disable anonymous access to Samba shares and apply least-privilege permissions.

5.) **Reduce unnecessary information disclosure**
Limit exposure of internal file paths, application structure, and sensitive resource locations.

6.) **Improve monitoring and visibility**
Monitor suspicious web requests, database abuse attempts, unusual directory browsing, and unauthorized file-share access.

7.) **Review service exposure regularly**
Conduct routine scans and configuration reviews to identify risky services and exposures before attackers do.

12) Lessons learned

This beginner-level project showed that offensive lab activity can be translated into useful defensive insight. Instead of treating the capstone only as a penetration test, reframing it from a blue-team perspective made the exercise more useful for understanding why the attacks succeeded and what defenders should improve.

Through this project, I learned how to:

- Identify the main stages of an attack path
- Recognize why basic security weaknesses are still dangerous
- Translate offensive activity into defender-focused findings
- Write a structured security analysis instead of a simple attack log
- Connect technical actions to practice remediation steps

This project also showed strong reporting matters. A completed lab becomes much more valuable when it is documented clearly and presented as a security analysis rather than only as a list of commands and results.

13) Disclaimer

- This project was completed in an authorized educational lab environment for training and portfolio purposes only. No unauthorized testing was performed against real-world systems or networks. The content in this report is intended to demonstrate beginner-level blue-team analysis and responsible cybersecurity documentation.